CSE 31
Computer Organization

Lecture 9 – C Structs

Announcements

- Lab
  - Lab 2 due this week (with 7 days grace period* after due date)
    - » Demo is REQUIRED to receive full credit**
  - Lab 3 out this week
    - » Due at 11:59pm on the same day of your next lab (with 7 days grace period* after due date)
    - » You must demo your submission to your TA within **14 days from posting of lab**
    - » Demo is REQUIRED to receive full credit**
- Reading and Homework assignments
  - Reading 01 (zyBooks 1.1 – 1.5) due 23-FEB
  - Homework 01 (zyBooks 1.1 – 1.5) due 02-MAR
    - » Complete Participation Activities in each section to receive grade towards Reading
    - » Complete Challenge Activities in each section to receive grade towards Homework
    - » IMPORTANT: Make sure to submit score to CatCourses by using the link provided on CatCourses

* There is a 10% penalty for late submissions during the grace period

** Submissions without demo will only receive partial credit for the TPS Activity portion of the assignment

Announcements

- Project 01
  - Due Fri, 20-MAR
  - Can work in teams of 2 students
    - » Each team member must identify teammate in "Comments…" text-box at the submission page
    - » If working in teams, each student must submit code (can be the same as teammate) and demo individually
    - » Grade can vary among teammates depending on demo
  - Demo required for project grade
    - » No partial credit for submission without demo
  - No grace period
    - » Must complete submission and demo by due date.

C structures : Overview

- A **struct** is a data structure composed from simpler data types or other **struct**s.
  - Like a class in Java/C++ but without methods or inheritance.

```
...
struct point {  // type definition
    int x;
    int y;
};

void PrintPoint(struct point p){

    printf("(%d, %d)", p.x, p.y);
}

int main() {
      struct point p1 = {0, 10}; // x = 0, y = 10
      PrintPoint(p1);
      ...
}
```

As always in C, the argument is passed by "value" – a copy is made.

C structures: Pointers to them

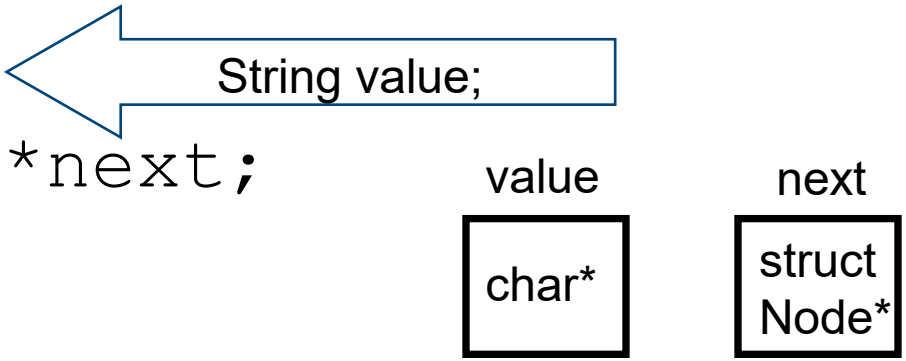
- Usually, more efficient to pass a pointer to the struct.
- The C arrow operator (->) dereferences and extracts a structure field (member) with a single operator.
- The following are equivalent:

```
struct point *p;
(*p).x = 7; // or p->x = 7;
printf("x is %d\n", (*p).x);
printf("x is %d\n", p->x);
```

Linked List Example

- Let's look at an example of using structures, pointers, `malloc()`, and `free()` to implement a linked list of strings.

```
/* node structure for linked list */
struct Node {
    char *value;
    struct Node *next;
};
```



How big are structs?

- Recall C operator `sizeof()` which gives size in bytes (of type or variable)

- How big is `sizeof(p)`?

```
struct p {
    char x;
    int y;
};
```

  – 5 bytes? 8 bytes?
  – Compiler may word align integer `y`
    » See https://en.wikipedia.org/wiki/Data_structure_alignment
  – More on this later lectures

typedef simplifies the code

```
struct Node {
    char *value;
    struct Node *next;
  };

/* "typedef" means define a new type */
typedef struct Node NodeStruct;
```

... OR ...

```
typedef struct Node {
    char *value;
    struct Node *next;
  } NodeStruct;
```

... THEN

```
typedef NodeStruct * List;
typedef char * String;
```

```
/* Note similarity! */
/* C++ */
/* To define 2 nodes */

struct Node {
    char *value;
    struct Node *next;
} node1, node2;
```

Linked List Example

```
/* Add a string to an existing list */
List cons(String s, List list)
{
  List node = (List) malloc(sizeof(NodeStruct));

  node->value = (String) malloc (strlen(s) + 1);
  strcpy(node->value, s);
  node->next = list;
  return node;
}
```

List is a `NodeStruct` pointer type

String is a char pointer type

```
typedef struct Node {
  char *value;
  struct Node *next;
} NodeStruct;
```

… THEN

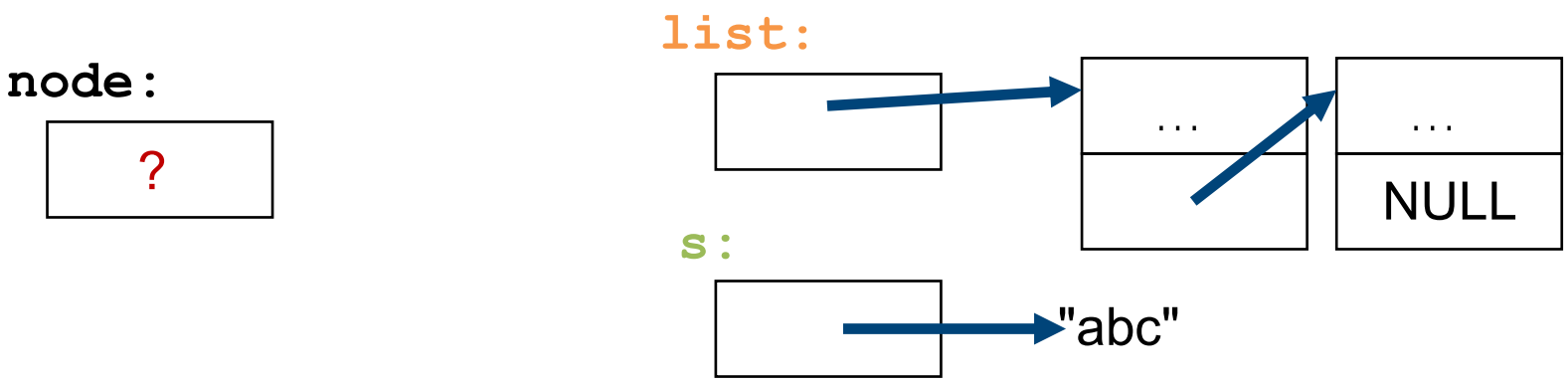
```
typedef NodeStruct * List;
typedef char * String;
```

- Somewhere in calling function…

```
String s1 = "abc", s2 = "cde";
List theList = NULL;
theList = cons(s2, theList);
theList = cons(s1, theList);
/* or embedded
theList = cons(s1, cons(s2, NULL)); */
```

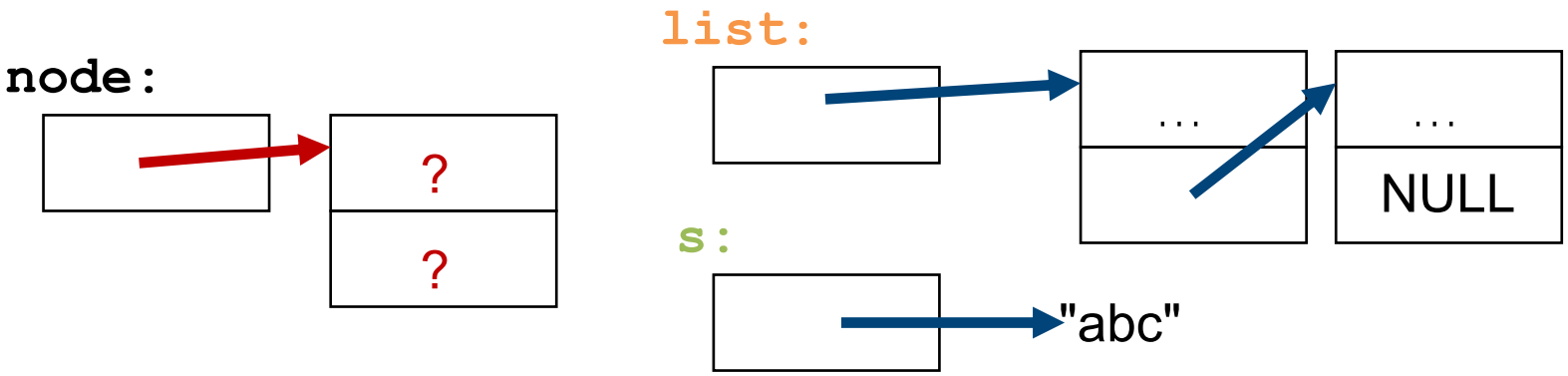
Linked List Example

```
/* Add a string to an existing list */
List cons(String s, List list)
{
  List node = (List) malloc(sizeof(NodeStruct));

  node->value = (String) malloc (strlen(s) + 1);
  strcpy(node->value, s);
  node->next = list;
  return node;
}
```



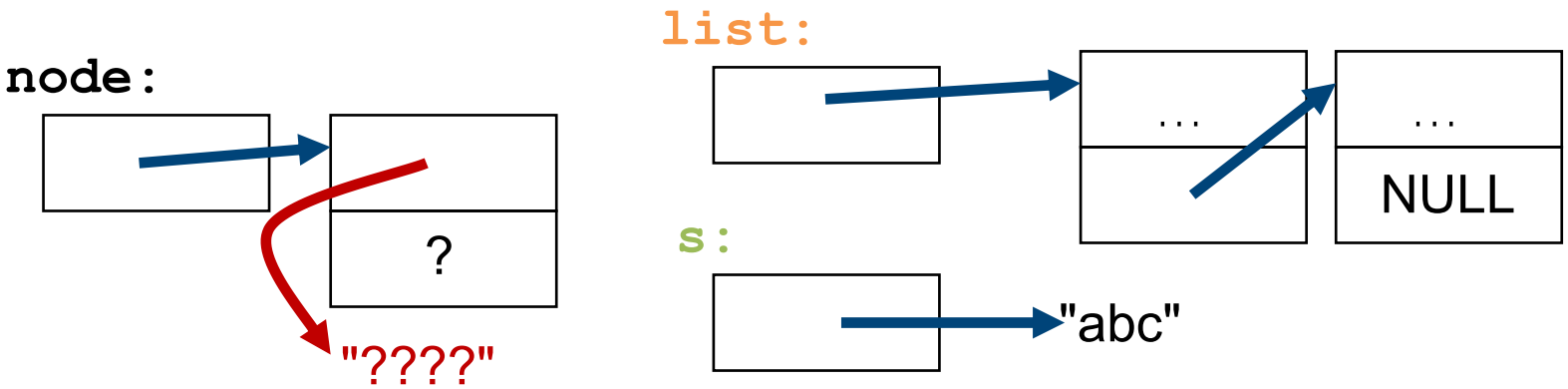
Linked List Example

```
/* Add a string to an existing list */
List cons(String s, List list)
{
  List node = (List) malloc(sizeof(NodeStruct));

  node->value = (String) malloc (strlen(s) + 1);
  strcpy(node->value, s);
  node->next = list;
  return node;
}
```



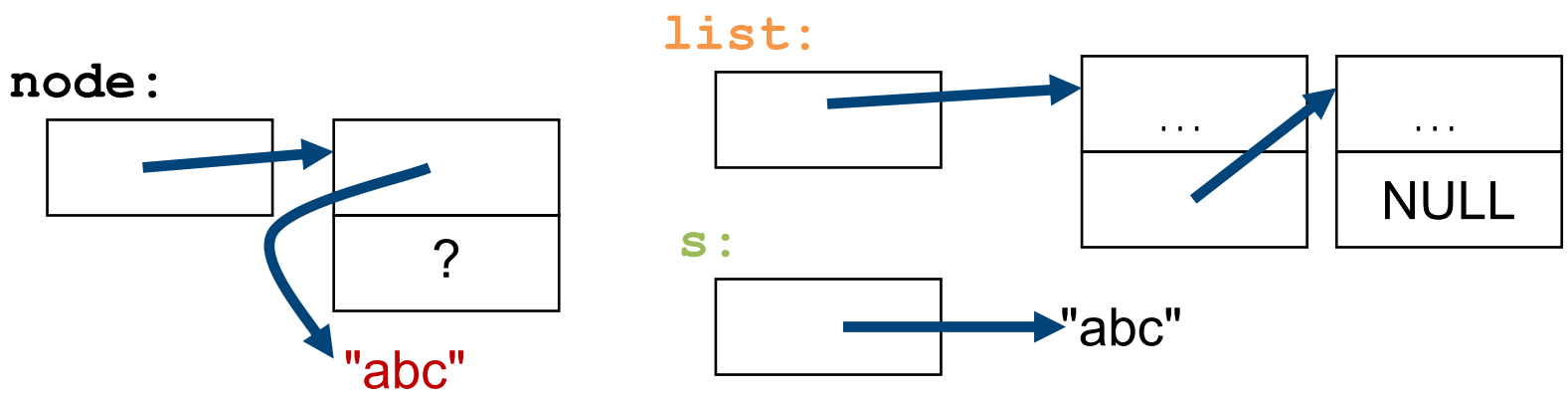
Linked List Example

```
/* Add a string to an existing list */
List cons(String s, List list)
{
  List node = (List) malloc(sizeof(NodeStruct));

  node->value = (String) malloc (strlen(s) + 1);
  strcpy(node->value, s);
  node->next = list;
  return node;
}
```



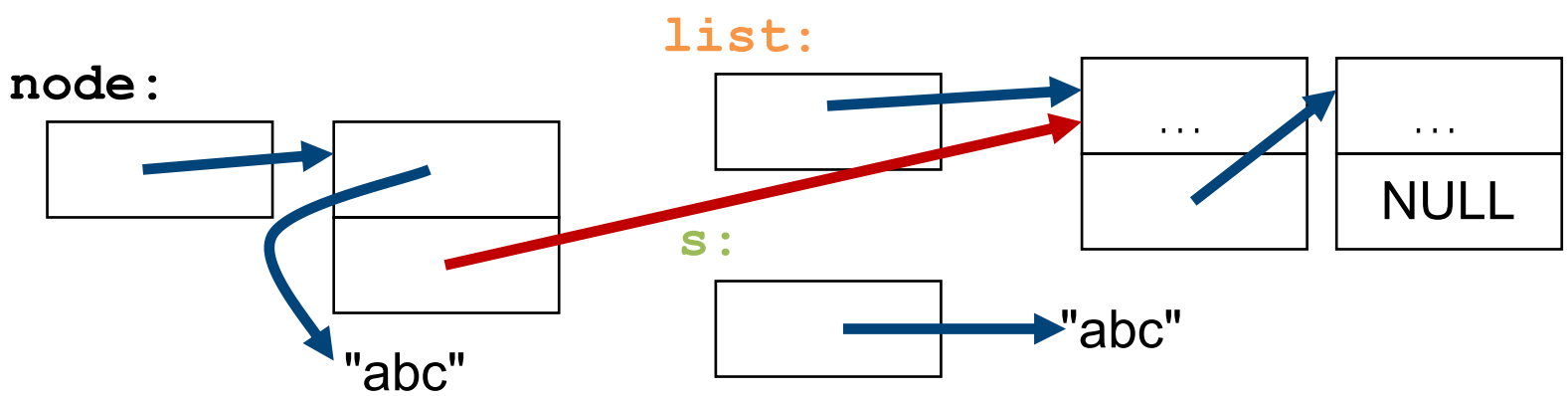
Linked List Example

```
/* Add a string to an existing list */
List cons(String s, List list)
{
  List node = (List) malloc(sizeof(NodeStruct));

  node->value = (String) malloc (strlen(s) + 1);
  strcpy(node->value, s);
  node->next = list;
  return node;
}
```



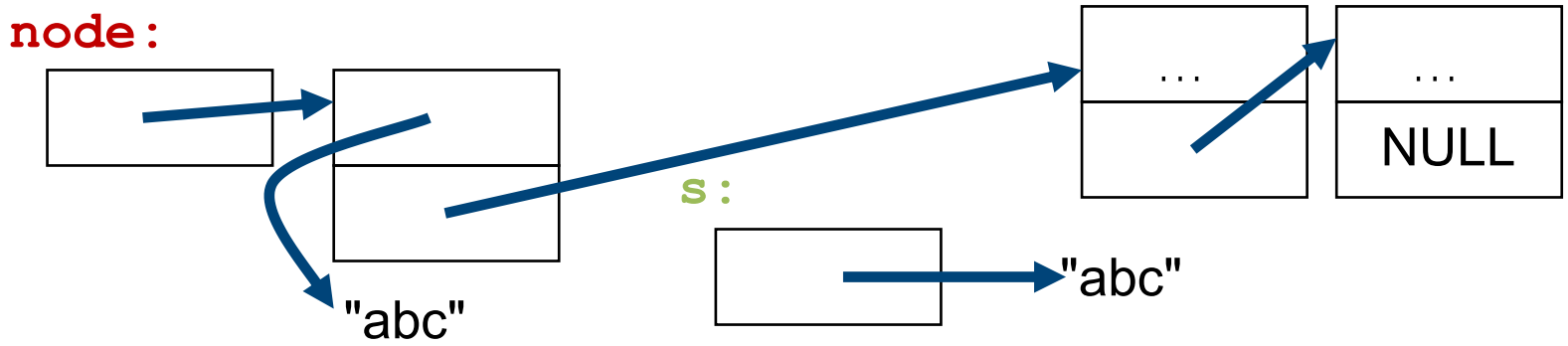
Linked List Example

```
/* Add a string to an existing list */
List cons(String s, List list)
{
  List node = (List) malloc(sizeof(NodeStruct));

  node->value = (String) malloc (strlen(s) + 1);
  strcpy(node->value, s);
  node->next = list;
  return node;
}
```



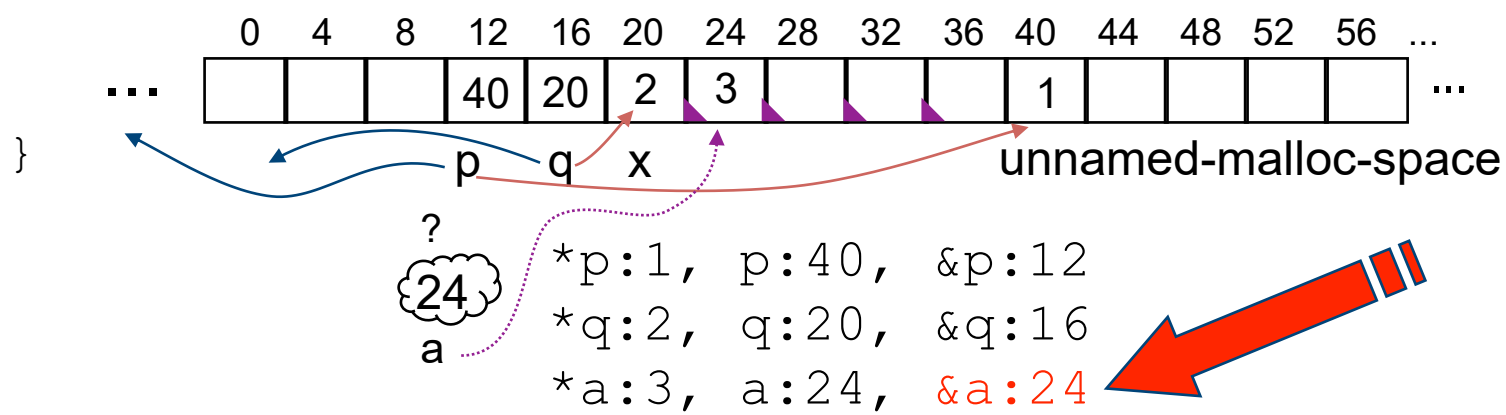
Linked List Example

```
/* Add a string to an existing list */
List cons(String s, List list)
{
  List node = (List) malloc(sizeof(NodeStruct));

  node->value = (String) malloc (strlen(s) + 1);
  strcpy(node->value, s);
  node->next = list;
  return node;
}
```



Revisiting Arrays

```
void foo() {
  int *p, *q, x;
  int a[4];
  p = (int *) malloc (sizeof(int));
  q = &x;

  *p = 1; // p[0] would also work here
  printf("*p:%u, p:%p, &p:%p\n", *p, p, &p);
  *q = 2; // q[0] would also work here
  printf("*q:%u, q:%p, &q:%p\n", *q, q, &q);
  *a = 3; // a[0] would also work here
  printf("*a:%u, a:%p, &a:%p\n", *a, a, &a);
}
```



K&R: "An array name is not a variable"

Don't forget the globals!

- Remember:
  - Structure declaration does not allocate memory
    - » Only when you instantiate it.
  - Variable declaration does allocate memory
- So far, we have talked about several different ways to allocate memory for data:
  1. Declaration of a local variable in a function (statically)

```
int i; struct Node list; char *string;
int ar[n];
```

  2. "Dynamic" allocation at runtime by calling allocation function (malloc).

```
ptr = (struct Node *) malloc(sizeof(struct Node)*n);
```

- One more possibility exists…
  3. Data declared outside of any procedure/function (i.e., before `main`).
  - Similar to #1 above but has "global" scope.

```
int myGlobal;
main() {
        …
}
```

Useful in C, but not in Java/C++

C you later… onto memory management!